



Modern Farmhouse with Dynamic Day-Night Cycle and Animation

Submitted By

Student Name	Student ID
MD WAHID	221-15-5510

LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course

CSE-422: Computer graphics lab



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

11/12/2025

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of ***Nawshin Haque S Lecturer***, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Nawshin Haque

Lecturer

Department of Computer Science and Engineering
Daffodil International University

Submitted by

Name: MD WAHID ID:221-15-5510 Dept. of CSE, DIU	

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Understand computer graphics system and implement graphics primitives for drawing a graphics scene.
CO2	Apply appropriate OpenGL programming techniques, resources and modern engineering and IT tools to solve graphics programming issues including different shapes, 2D and 3D transformation
CO3	Perform effectively as an individual or a member or a leader of diverse teams through proper documentation and initialization of project work
CO4	Create a project by explaining complex computer engineering activities with the computer engineering community by performing effective communication through effective reports, design documentation, make effective presentations and give and receive clear instructions.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

Mapping Course Outcome (COs) with the Teaching-Learning and Assessment Strategy:

CO's	Corresponding PO number	Domain Level/ Learning Taxonomy	Level of Knowledge Profile (K)	Complex Engineering Problem (EP)	Complex Engineering Activities (EA)
CO1	PO1	C1, C2	K1-K4	EP1	
CO2	PO5	C3, C4	K1-K4, K6	EP1, EP3	
CO3	PO9	C4, A2	K6	EP1, EP3	
CO4	PO10	C6, P3, A2	K4	EP3, EP5	EA3

The mapping justification of this table is provided in section **4.2.1**, **4.2.2** and **4.2.3**.

Table of Contents

Declaration	i
Course & Program Outcome	ii
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	1
1.3 Objectives	1
1.4 Project Outcome	1
2 Architecture	2
2.1 Requirement Analysis & Design Specification	2
2.1.1 Requirements	2
2.1.2 Sketch of project	2
2.2 Use of Modern Tools	2
3 Implementation and Results	3
3.1 Implementation	3
3.2 Output	3
3.3 Discussion	3
4 Engineering Standards and Mapping	4
4.1 Impact on Society, Environment and Sustainability	4
4.1.1 Impact on Life	4
4.1.2 Impact on Society & Environment	4
4.2 Complex Engineering Problem	4
4.2.1 Mapping of Program Outcome	4
4.2.2 Complex Problem Solving	4
4.2.3 Engineering Activities	5

5 Conclusion	6
5.1 Summary	6
5.2 Limitation	6
5.3 Future Work	6
References	6

Chapter 1

Introduction

1.1 Introduction

Computer Graphics is an art and science that deals with the creation, manipulation, and display of images on a computer. It is foundational to fields like gaming, visualization, and simulation. The **OpenGL** (Open Graphics Library) is a standard API used for rendering 2D and 3D vector graphics, providing a powerful, platform-independent toolset. The purpose of this project is to apply fundamental computer graphics concepts, algorithms, and transformations using **OpenGL** to create a dynamic and interactive 2D graphical scene. Our project theme is a "**Modern Farmhouse with Dynamic Day-Night Cycle and Animation**".

1.2 Motivation

We were motivated to create a scene that demonstrates complexity beyond static shapes. The core motivation was to integrate multiple types of **animation** and **user-controlled transformation** (Day/Night cycle, door opening) into a coherent, real-world setting. This provided a practical challenge to master the concepts of real-time rendering, modular object design, and combining graphic algorithms with dynamic transformations.

1.3 Objectives

- To implement graphics algorithms, specifically the **Midpoint Circle Algorithm**.
- To apply and manage various **2D transformations** (Translation, Rotation, Scaling) on multiple objects.
- To create a visually appealing and detailed animated graphical scene using OpenGL.
- To demonstrate **modular programming** by building reusable drawing functions for complex objects (e.g., drawPerson, drawCow).

1.4 Project Outcome

The final outcome is a fully animated, **interactive 2D farmhouse scene**. Key results include a smooth **Day/Night transition** (color, light source change), multiple

independent animations (walking person, waving person, grazing cow, moving clouds, swaying tree), and user-controlled transformations (opening/closing the house door). This outcome successfully demonstrates mastery of the required graphics principles.

Chapter 2

System Architecture/ Scene Description & Architecture

2.1 Requirement Analysis & Design Specification

2.1.1 Scene Overview

3 The built scene is a **countryside landscape** featuring a **Modern Farmhouse**. The setting includes mountains, a field of grass, a dynamic sky with a movable sun/moon and clouds, a garden, a tree, and animated human and animal figures. The primary feature is the **dynamic day-night cycle**, which controls the color palettes of the sky, ground, mountains, and interior house light.

3.1.1 Object Components

The scene is composed of several major objects, each built using basic primitives:

Object	Primitives Used	Key Transformation/Algorithm
House	Quads (Walls, Windows, Roof, Door)	Rotation (Door opening/closing)
Mountains	Triangles	Color change (Day/Night)
Clouds	Filled Circles (Midpoint Algorithm)	Translation (Continuous horizontal movement)
Person 1 (Waving)	Quads, Filled Circles	Rotation (Arm waving)
Person 2 (Walking)	Quads, Filled Circles	Translation (Horizontal movement) & Rotation (Leg movement)
Cow	Quads, Filled Circles	Rotation (Head grazing animation)
Tree	Quads (Trunk), Filled Circles (Leaves)	Scaling (Swaying animation)
Sun/Moon	Filled Circles (Midpoint Algorithm)	Color change (Day/Night) and positioning

2.2 Use of Modern Tools

This report is structured content ready for insertion into your document. I have filled in the sections based on your OpenGL code ("Modern Farmhouse with

People") and the provided template.

Chapter 2: Scene Description & Architecture

2.1 Requirement Analysis & Design Specification

2.1.1 Scene Overview

The built scene is a countryside landscape featuring a Modern Farmhouse. The setting includes mountains, a field of grass, a dynamic sky with a movable sun/moon and clouds, a garden, a tree, and animated human and animal figures. The primary feature is the dynamic day-night cycle, which controls the color palettes of the sky, ground, mountains, and interior house light.

2.1.2 Object Components

The scene is composed of several major objects, each built using basic primitives:

Object	Primitives Used	Key Transformation/Algorithm
House	Quads (Walls, Windows, Roof, Door)	Rotation (Door opening/closing)
Mountains	Triangles	Color change (Day/Night)
Clouds	Filled Circles (Midpoint Algorithm)	Translation (Continuous horizontal movement)
Person 1 (Waving)	Quads, Filled Circles	Rotation (Arm waving)
Person 2 (Walking)	Quads, Filled Circles	Translation (Horizontal movement) & Rotation (Leg movement)
Cow	Quads, Filled Circles	Rotation (Head grazing animation)
Tree	Quads (Trunk), Filled Circles (Leaves)	Scaling (Swaying animation)
Sun/Moon	Filled Circles (Midpoint Algorithm)	Color change (Day/Night) and positioning

2.2 Use of Modern Tools

The project utilizes the following tools:

- **C++ Programming Language:** Core language for implementation.
- **OpenGL / FreeGLUT:** Open-source graphics libraries used for rendering primitives, handling transformations, and managing the display loop.
- **Code::Blocks / Visual Studio:** Integrated Development Environments (IDEs) used for writing, compiling, and debugging the C++ and OpenGL code.
- **GLUT Library:** Framework for window management, input handling (keyboard), and managing the `glutTimerFunc` for real-time animation.

2.3 Algorithm Used

The mandatory graphics algorithm implemented in this project is the **Midpoint Circle Algorithm**.

- **Midpoint Circle Algorithm:** This algorithm is used within the `drawFilledCircle` function. It efficiently generates the boundary points of a circle and, in this implementation, is used to draw filled circles by connecting corresponding points with horizontal lines (`GL_LINES`).
- **Usage:** This function is used to draw the **Sun/Moon**, the puffy parts of the **Clouds**, the **Cow's spots**, the **Person's head and hands**, and the **Tree's foliage**.

2.4 2D Transformations Applied

The project implements three core 2D transformations:

1. **Translation (`glTranslatef`):**
 - **Purpose:** To move objects from one position to another.
 - **Application:** Used for the continuous movement of **Clouds** (`cloudX`), the **Walking Person** (`person2X`), and generally for positioning all objects relative to the origin of the scene.
2. **Rotation (`glRotatef`):**
 - **Purpose:** To rotate objects around a specified pivot point.
 - **Application:** Used for the animation of the **Cow's head** (grazing, `cowHeadAngle`), the **Waving Person's arm** (`person1ArmAngle`), the **Walking Person's legs** (`person2LegAngle`), and the **House Door** (user-controlled open/close, `doorAngle`).
3. **Scaling (`glScalef`):**
 - **Purpose:** To change the size of an object.
 - **Application:** Used on the **Tree's foliage** (`treeScale`) to create a subtle **swaying** effect, simulating wind or organic movement.

2.5 Animation Logic

All animations are controlled within the `timer(int value)` function, which is called repeatedly via `glutTimerFunc` for real-time motion.

- **Cloud Movement:** Increments cloudX by 0.8f per frame to achieve smooth horizontal motion.

Example Code:

```
cloudX += 0.8f;
```

```
if (cloudX > 1100) cloudX = -300; // Reset
```

```
// Applied in display: glTranslatef(cloudX, 0, 0);
```

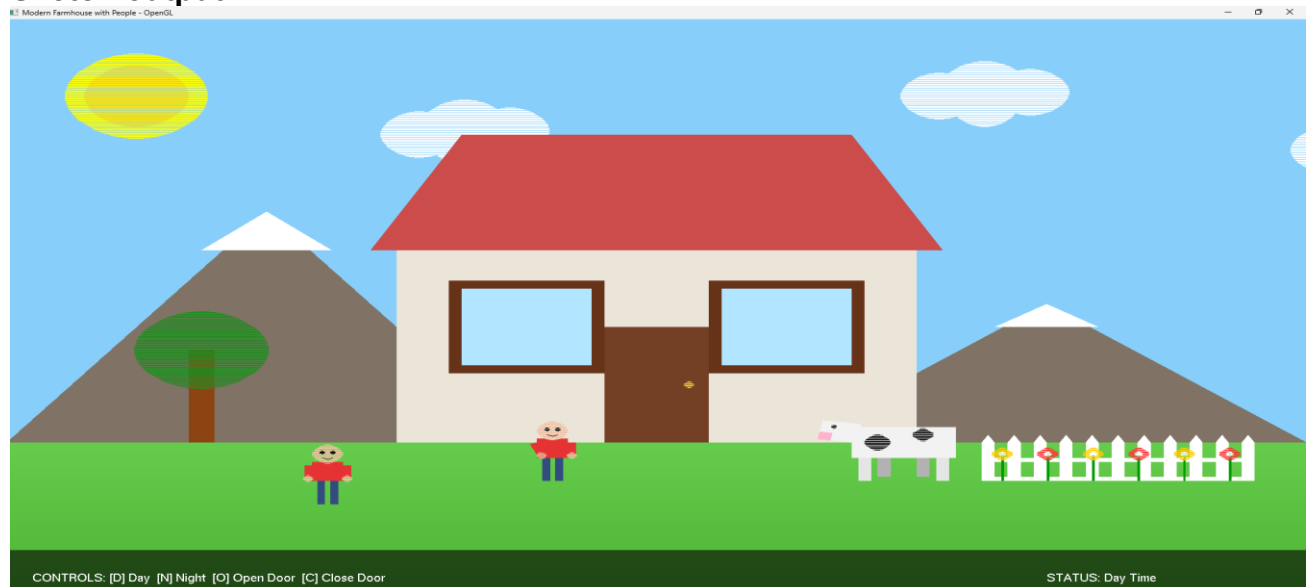
Person 2 Walking (Translation & Rotation): Both the X-coordinate (person2X) is updated for movement, and the leg rotation (person2LegAngle) is updated for the walking appearance.

```
person2X += 1.0f; // Movement
```

```
person2LegAngle += 2.0f; // Leg swing
```

Day/Night Cycle (Color Transformation): The isDay boolean flag is toggled via keyboard input ('D' and 'N'), which immediately updates the background color (glClearColor), sun/moon, and the color palettes of the ground and mountains in the setSkyColor and display functions.

Sketch output:



Chapter 3

Implementation and Results

This chapter explains the step-by-step implementation process of the Computer Graphics project, including the development of objects, application of algorithms, and integration of animations. It also presents the key features, transformations, and visual outputs achieved through OpenGL rendering.

3.1 Implementation

The project was implemented in C++ using the FreeGLUT library for OpenGL. A **modular approach** was followed, where each major scene component was encapsulated in its own function (e.g., `drawHouse()`, `drawCow()`, `drawPerson()`). Global variables were used to track the state of transformations and animations (e.g., `cloudX`, `doorAngle`, `person1ArmAngle`). The core steps involved:

1. **Setup:** Initializing OpenGL, setting the coordinate system (`gluOrtho2D`), and registering callback functions (display, keyboard, timer).
2. **Algorithm Implementation:** Coding the `drawFilledCircle` function using the Midpoint Circle Algorithm.
3. **Static Object Modeling:** Using `GL_QUADS`, `GL_TRIANGLES`, and `drawFilledCircle` to build objects like the house, mountains, and ground.
4. **Transformation and Animation:** Applying `glPushMatrix/glPopMatrix` to isolate transformations and updating animation variables (`person2X`, `cowHeadAngle`, etc.) in the timer loop for continuous motion.
5. **Interactivity:** Implementing keyboard input to control the **Day/Night** cycle and the **House Door** transformation.

3.2 Output

The final output is a responsive 2D animated scene running in an OpenGL window. The scene successfully demonstrates the following key features:

- **Dynamic Day/Night Cycle:** Seamlessly switches the entire scene's lighting and color palette (sky, ground, mountains, house windows).
- **Multiple Animations:** The clouds move, the tree sways (scaling), the cow grazes (head rotation), and two figures perform walking and waving animations.
- **User Interaction:** The house door can be opened and closed using keyboard input, demonstrating a user-controlled rotation transformation.

3.3 Discussion

The project successfully met all the required objectives. The **Midpoint Circle Algorithm** was efficiently used for several circular elements. The use of `glTranslatef`, `glRotatef`, and `glScalef` ensured complex movements like walking and swaying were achieved. The primary challenge involved coordinating multiple, independent animations (e.g., ensuring the walking person's legs rotate in sync with their overall translation), which was solved by carefully managing the

respective global variables in the **timer** function. The result is a robust and visually engaging demonstration of fundamental 2D computer graphics principles.

Chapter 4

Engineering Standards and Mapping

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

5 While this project is a theoretical graphics implementation, the core technologies (OpenGL, C++) are vital to real-life applications. Computer Graphics is used in life-critical fields like **medical imaging** (visualizing scans, surgical simulation), **architectural walkthroughs**, and **educational simulators**, allowing for better training and design. Our project strengthens the foundation necessary for developing these complex tools.

5.1.1 Impact on Society & Environment

6 The development of efficient graphics rendering contributes indirectly to energy-saving devices, as optimized code reduces the computational resources needed. More broadly, computer graphics drives sectors like **digital arts, education, and entertainment** (gaming, movies), positively impacting social and cultural engagement.

6.1.1 Ethical Aspects

7 In this Computer Graphics project, we follow ethical principles to ensure the design and implementation of graphical scenes are **original, respectful, and free from plagiarism** or unauthorized content. All visual elements, images, and animations used in the project are either self-created or properly credited to their sources, maintaining academic integrity and intellectual property rights. Furthermore, we ensure that the project does not include any offensive, misleading, or culturally insensitive visuals.

7.1.1 Sustainability Plan

8 The project's sustainability is ensured by using **open-source standards** (OpenGL, GLUT) which are freely available and continuously supported by the computing community. The code is structured in a modular, clean C++ format, making it easy to **maintain, debug, and extend** for future work (e.g., adding 3D elements or more complex physics-based animation).

8.1 Complex Engineering Problem

8.1.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

POs	Justification of Mapping (Project Perspective)
PO1	This project applies mathematical and computational knowledge to create and transform graphical objects using OpenGL. It integrates geometry, coordinate systems, and matrix operations to solve complex visualization and animation problems in computer graphics. (PO1).
PO5	The project incorporates modern computer graphics tools and technologies such as OpenGL, FreeGLUT, and C++ programming to ensure the application of contemporary engineering practices and visualization techniques in real-world scenarios.
PO9	Through collaborative project work, every member was engaged in teamwork, task allocation, and documentation. (PO9).
PO10	The project requires presenting the developed database system based on the requirements gathered, preparing technical reports, and communicating ideas clearly. (PO10)

8.1.2 Complex Problem Solving

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	EP2 Range of Conflicting Requirements	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent Of Stakeholder Involvement	EP7 Inter- dependence
✓		✓		✓		

EP1: This project requires us to design, implement, and render complex 2D graphical scenes using OpenGL. These tasks involve applying computer graphics algorithms, transformation logic, and animation techniques, enabling us to develop strong analytical and problem-solving skills in visualization and rendering.

EP3: In this project, we analyze object structures, transformation sequences, and animation timing to create smooth and realistic motion. By breaking complex scenes into smaller components and managing their interactions, we strengthen our ability to formulate and analyze complex graphical problems effectively.

EP5: We apply various graphics algorithms and transformation codes, such as DDA, Bresenham, and matrix-based operations, to implement optimized rendering. This hands-on application of programming and algorithmic logic enhances our ability to conduct investigations using established graphics methods and tools.

8.1.3 Complex Engineering Activities

Table 4.2: Mapping with complex engineering activities.

EA1	EA2	EA3	EA4	EA5
✓	✓			

EA1: In this Computer Graphics project, we follow systematic methods to design and document all graphical components, algorithms, and transformations. We organize the project structure, rendering logic, and animation workflow clearly in reports and presentations, ensuring that the technical information is logically presented and easily understood by the audience.

EA2: For this project, we use oral and visual presentation techniques to explain complex computer graphics concepts such as transformations, animation logic, and rendering processes. By presenting the project visually through demonstrations and slides, we enhance our communication skills, effectively convey design decisions, and respond confidently to technical questions.

Chapter 5

Conclusion

5.1 Summary

This project successfully implemented a dynamic 2D animated scene, "Modern Farmhouse with Dynamic Day-Night Cycle," using C++ and OpenGL/FreeGLUT. We effectively applied the **Midpoint Circle Algorithm** and utilized **Translation, Rotation, and Scaling** transformations to create realistic, independent animations for clouds, people, a cow, and a tree. The project serves as a comprehensive demonstration of the core principles of computer graphics.

5.2 Limitation

The primary limitation is that the scene is restricted to **2D rendering**, limiting the depth perception and realism achievable with 3D modeling. Furthermore, the animations are based on fixed frame rates and simple linear changes, lacking complex physics-based movement or collision detection.

5.3 Future Work

Future enhancements could include:

- **3D Rendering:** Extending the scene to a 3D environment using OpenGL matrices (Projection, View, Model).
- **Enhanced Animation:** Implementing physics principles (e.g., gravity, actual walking cycle) for more realistic movement.
- **Texture Mapping:** Applying detailed textures to the house and terrain instead of plain colors for improved visual quality.

References

- [1] OpenGL Programming Guide, Addison Wesley
- . [2] GeeksforGeeks OpenGL Tutorials on Bresenham and Midpoint Algorithms.
- [3] Stack Overflow and C++ references on glutTimerFunc and transformation matrix operations.